

0 Cleaner

Multi-Tenant SaaS Platform

Backend Architecture & Development Plan

1. System Overview

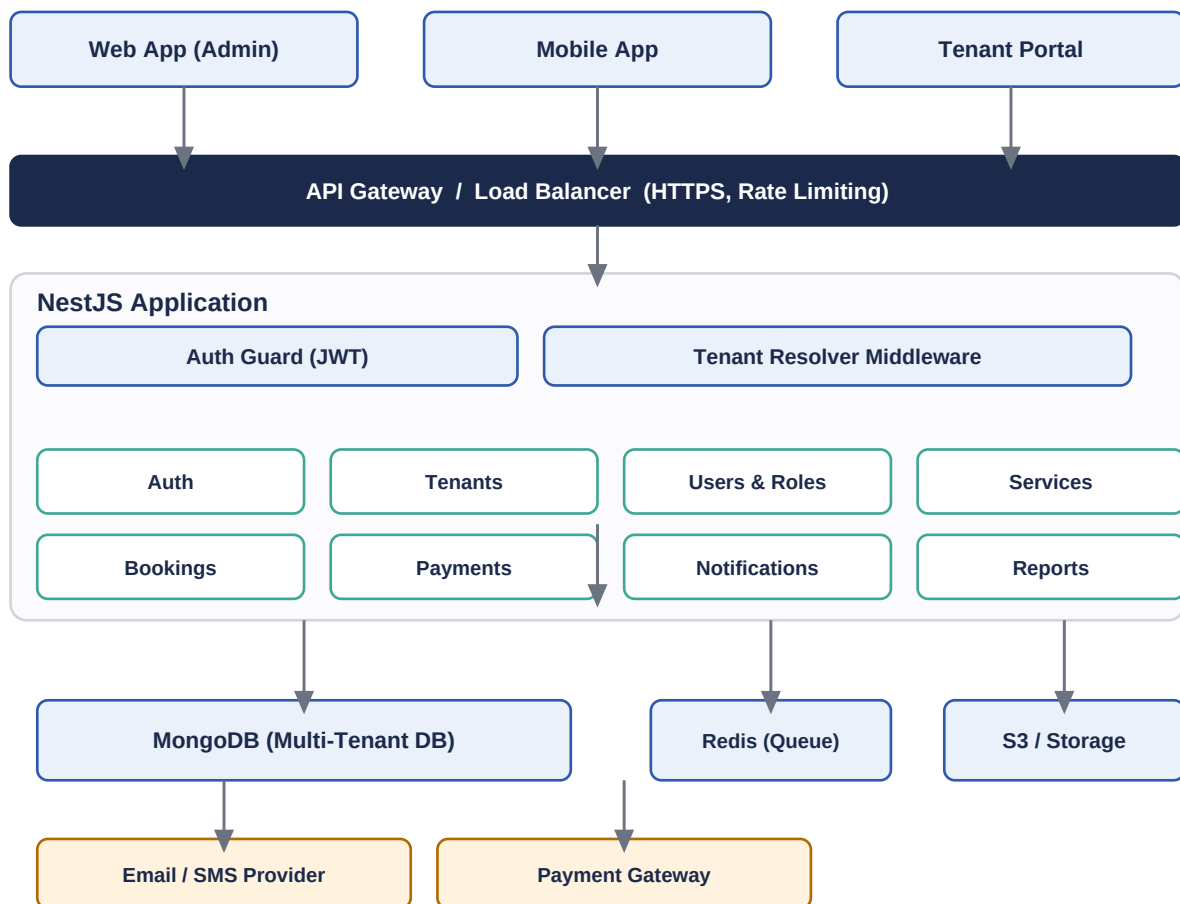
0 Cleaner is a multi-tenant SaaS platform sold to different cleaning companies. Each cleaning company that signs up becomes a **tenant**, with its own isolated data (branches, staff, customers, bookings, invoices) inside a shared application and database.

This document lays out the target backend architecture, the database structure, the authentication flow already implemented, and the remaining modules with their endpoints, grouped into build phases so the team can track progress module by module.

Current status

Area	Status	Notes
Tenant module	✓ Done	Create / List / Get by ID / Update / Delete
Authentication module	✓ Done	Login, Signup, Refresh Token, Logout
Everything else (Users, Roles, Services, Bookings, Payments, Notifications, Reports)	■ To build	Planned — see phases below

2. High-Level Architecture



Clients (web admin panel, mobile app, tenant-facing portal) talk to a single API gateway over HTTPS. Every request passes through an authentication guard (validates the JWT) and a tenant-resolver middleware (extracts the tenant from the token or subdomain and scopes every database query to that tenant). Feature modules stay decoupled and each owns its own controllers, services and repositories.

Core building blocks

- **API Gateway:** single entry point, handles HTTPS termination, rate limiting and routing.
- **Auth Guard:** validates the JWT access token on every protected route.
- **Tenant Resolver:** reads the tenant id from the JWT payload (or subdomain) and injects it into the request context so every query is automatically scoped to that tenant.
- **Feature modules:** Auth, Tenants, Users & Roles, Services, Bookings, Payments, Notifications, Reports — each a self-contained NestJS module.
- **MongoDB:** single shared database (via Mongoose), tenant isolation enforced at the document level via a `tenantId` field on every tenant-owned collection.
- **Redis:** caching, session/refresh-token storage, and background job queues (BullMQ).
- **External services:** email/SMS provider for notifications, payment gateway for billing.

3. Multi-Tenancy Strategy

There are three common ways to isolate tenant data. The table below compares them so the choice is documented and can be revisited later.

Strategy	Isolation	Cost / Complexity	Fit for 0 Cleaner
Shared collections (tenantId field)	Logical, enforced in the application/ODM layer	Low — one set of collections, simplest to run and back up	Recommended for launch: simplest to build and operate at moderate scale
Database-per-tenant	Stronger, separate MongoDB database per tenant	Medium/High — one connection + backup policy per tenant	Good fallback if a large tenant needs stronger isolation later
Cluster-per-tenant	Strongest, full physical isolation	High — one cluster to provision/monitor per tenant	Overkill for now; consider for enterprise-tier tenants only

Decision: start with the shared-collections strategy — a **tenantId** field on every tenant-owned document, plus a Mongoose plugin/pre-hook applied to every schema that automatically adds **{ tenantId }** to every query and injects it on every insert, so a developer can't accidentally forget the filter. A unit test suite specifically checks that no query can return another tenant's documents.

4. Tech Stack

Layer	Choice	Why
Runtime / Framework	Node.js + NestJS	Modular, opinionated structure (modules/controllers/services), built-in DI, guards and pipes — fits a multi-tenant SaaS well
Language	TypeScript	Type safety across DTOs, schemas and tenant-scoped services
Database	MongoDB	Flexible document model, fast to iterate on while the schema is still evolving, easy horizontal scaling later
ODM	Mongoose	Schema validation, and a shared plugin/pre-hook that injects the tenant-scoping filter into every query
Cache / Queue	Redis + BullMQ	Refresh-token/session storage, rate limiting, background jobs (emails, reminders)
Auth	JWT (access + refresh)	Stateless access token, rotated refresh token stored server-side (Redis/DB)
File storage	S3-compatible bucket	Tenant logos, before/after job photos, invoices/PDF exports
Infra	Docker + CI/CD	Reproducible environments, automated tests and deploys
Docs	Swagger / OpenAPI	Auto-generated API reference for the frontend/mobile teams

5. Core Database Schema

Every collection below (except **tenants** itself) carries a **tenantId** reference so it can be scoped per tenant. Field names match the actual Mongoose schema style used in the codebase (camelCase, **_id** as the Mongo-generated primary key).

Collection	Key fields	Relationships
tenants	_id, name, slug, planId, status	referenced by: users, branches, customers, bookings...
users	_id, tenantId, email, passwordHash, roleId	references tenant; references role
roles / permissions	_id, tenantId, name, permissions[]	many-to-many users ↔ permissions
plans / subscriptions	_id, tenantId, planName, status, renewsAt	references tenant; drives feature limits/billing
branches	_id, tenantId, name, address	references tenant; referenced by staff/bookings
staff	_id, tenantId, userId, branchId, skills	references user + branch; assigned to bookings
customers	_id, tenantId, name, phone, address	references tenant; referenced by bookings
services	_id, tenantId, name, basePrice, duration	references tenant; used by bookings
bookings / jobs	_id, tenantId, customerId, staffId, serviceId, status, scheduledAt	central entity — links customer, staff, service
payments / invoices	_id, tenantId, bookingId, amount, status	references booking; drives reporting
notifications	_id, tenantId, userId, type, payload, readAt	references tenant + user

6. Authentication & Tenant Resolution Flow

The Authentication module is already built. This is the flow it implements end to end:

- **Signup:** a tenant admin registers → creates the tenant record and the first admin user in one transaction.
- **Login:** validates credentials, issues a short-lived JWT **access token** (contains `userId`, `tenantId`, `role`) and a longer-lived **refresh token** (stored hashed in Redis/DB, rotated on use).
- **Authenticated requests:** the Auth Guard verifies the access token; the Tenant Resolver reads **tenantId** from the token payload and attaches it to the request context for every downstream query.
- **Refresh Token:** when the access token expires, the client calls this endpoint with the refresh token to get a new access/refresh pair; the old refresh token is invalidated (rotation).
- **Logout:** invalidates the current refresh token server-side so it can't be reused.

Hardening to add: per-tenant rate limiting on login/signup, account lockout after repeated failed attempts, and role/permission claims embedded in the access token so guards can check authorization without an extra DB round-trip.

7. Module Roadmap

Suggested build order — each phase is deployable and testable on its own before moving to the next.

Phase 0 — Foundations (done)

Tenant management and authentication — already implemented.

Module	Endpoints	Status
Tenant	POST / GET / GET :id / PATCH / DELETE	✓ Done
Authentication	POST login / signup / refresh-token / logout	✓ Done

Phase 1 — Users, Roles & Permissions

Let each tenant manage its own staff accounts and access levels (RBAC).

Module	Endpoints	Status
Users	POST /users · GET /users · GET /users/:id · PATCH /users/:id · DELETE /users/:id · PATCH /users/:id/status	■ To build
Roles & Permissions	POST /roles · GET /roles · PATCH /roles/:id · DELETE /roles/:id · GET /permissions	■ To build

Phase 2 — Subscriptions & Billing Plans

Control what each tenant can access based on their plan.

Module	Endpoints	Status
Plans	GET /plans · POST /plans (admin) · PATCH /plans/:id	■ To build
Subscriptions	POST /subscriptions · GET /subscriptions/current · PATCH /subscriptions/:id/cancel	■ To build

Phase 3 — Core Domain — Branches, Services, Staff, Customers

The operational data every cleaning company needs before it can take bookings.

Module	Endpoints	Status
Branches	Full CRUD: POST / GET / GET :id / PATCH / DELETE	■ To build
Services	Full CRUD + GET /services/:id/price	■ To build
Staff	Full CRUD + PATCH /staff/:id/availability	■ To build
Customers	Full CRUD + GET /customers/:id/bookings	■ To build

Phase 4 — Bookings / Jobs Engine

The core workflow: scheduling, assigning staff, tracking job status.

Module	Endpoints	Status
Bookings	POST /bookings · GET /bookings · GET /bookings/:id · PATCH /bookings/:id · PATCH /bookings/:id/assign-staff · PATCH /bookings/:id/status · DELETE /bookings/:id	■ To build
Calendar	GET /calendar?branch_id=&date=	■ To build

Phase 5 — Payments & Invoicing

Close the loop from a completed job to a paid invoice.

Module	Endpoints	Status
Invoices	POST /invoices · GET /invoices · GET /invoices/:id · GET /invoices/:id/pdf	■ To build
Payments	POST /payments · POST /payments/webhook · GET /payments/:id	■ To build

Phase 6 — Notifications & Reporting

Keep tenants informed and give them visibility into their business.

Module	Endpoints	Status
Notifications	GET /notifications · PATCH /notifications/:id/read · POST /notifications/preferences	■ To build
Reports / Dashboard	GET /reports/revenue · GET /reports/bookings · GET /dashboard/summary	■ To build

Phase 7 — Hardening & Launch

Make the platform production-ready.

- Automated tests (unit + e2e) covering tenant isolation specifically.
- Audit logging for sensitive actions (role changes, deletions, payments).
- Rate limiting, input validation and centralized error handling across all modules.
- CI/CD pipeline, staging environment, and monitoring/alerting before go-live.

8. Security Considerations

- **Tenant isolation:** never trust a tenantId coming from the request body — always derive it from the authenticated JWT payload.
- **Password storage:** bcrypt/argon2 hashing, never plain text or reversible encryption.
- **Refresh token rotation:** invalidate the old refresh token every time a new one is issued; detect and react to reuse of a revoked token (possible theft).
- **RBAC:** guard every endpoint with both an authentication check and a permission check (e.g. only tenant admins can delete staff).
- **Rate limiting:** especially on `/auth/login` and `/auth/signup` to prevent brute-force and abuse.
- **Input validation:** DTOs with class-validator on every endpoint; reject unknown fields.
- **Secrets management:** environment variables / secret manager, never committed to git.
- **Audit trail:** log who changed what and when, per tenant.

9. Deployment Plan

- **Containerization:** Dockerfile for the NestJS app; docker-compose for local dev (app + MongoDB + Redis).
- **Environments:** development → staging → production, each with its own database and env vars.
- **CI/CD:** run lint + tests on every push; auto-deploy staging on merge to main; manual promotion to production.
- **Migrations:** run automatically as part of the deploy pipeline, never manually in production.
- **Monitoring:** structured logging, error tracking (e.g. Sentry), and uptime/health checks per environment.

10. Immediate Next Steps

- Finalize the database schema for Phase 1 (Users, Roles, Permissions) and write migrations.
- Implement the tenant-scoping mechanism (global query filter) once, before adding more modules — every module after this depends on it.
- Build the Users & Roles module end to end, then move to Branches/Services/Staff.
- Add Swagger docs as each module ships so the frontend team can integrate in parallel.